

Practical Question 1 — Event-Driven File Processing (GCP)

Scenario : Implement an automated system that processes files uploaded to Cloud Storage and records metadata in a database.

Implementation:

I will first create a Cloud Storage bucket named arun-file-upload-lab to act as the source. Whenever a file is uploaded to this bucket, a Cloud Function (Gen 2) will be triggered automatically. The function will read the file name and size from the event metadata. It will then store this information as a document in Cloud Firestore. Finally, the processed file details will be logged in Cloud Logging for monitoring and verification.

STEP 1: Set Project & Enable Required APIs

i) Set your project

```
gcloud config set project proven-record-481104-g3
```

ii) Enable required services

```
gcloud services enable \  
cloudfunctions.googleapis.com \  
eventarc.googleapis.com \  
storage.googleapis.com \  
firestore.googleapis.com \  
logging.googleapis.com
```

STEP 2: Create Cloud Storage Bucket

Create bucket

```
gsutil mb -l asia-south2 gs://arun-file-upload-lab
```

This bucket is where files will be uploaded and events will be generated.

STEP 3: Create Firestore Database

Enable Firestore (Native mode)

```
firestore databases create arundb  
--region=asia-south2  
--type=firestore-native
```

STEP 4: Write Cloud Function (Gen 2)

Create a folder

```
mkdir Handson1
cd Handson1
```

main.py (Function Code)

```
from google.cloud import firestore
import logging

# Firestore client
db = firestore.Client()

def arun_upload_file(event, context):
    """
    Triggered when a file is uploaded to Cloud Storage
    """

    file_name = event['name']
    file_size = event['size']
    bucket_name = event['bucket']

    # Prepare data
    file_data = {
        "file_name": file_name,
        "file_size": int(file_size),
        "bucket": bucket_name
    }

    # Insert into Firestore
    db.collection("arun_users_activity").add(file_data)

    # Log to Cloud Logging
    logging.info(
        f"Processed file: {file_name}, Size: {file_size} bytes, Bucket: {bucket_name}"
    )
```

requirements.txt

```
google-cloud-firestore
```

STEP 5: Deploy Cloud Function (Gen 2)

Deployment Command

```

gcloud functions deploy arun_upload_file
--gen2
--runtime=python311
--region=asia-south2
--source=.
--entry-point=arun_upload_file
--trigger-event-filters="type=google.cloud.storage.object.v1.finalized"
--trigger-event-filters="bucket=arun-file-upload-lab"

```

The screenshot shows a VS Code editor with a file named `main.py` containing a Python function `arun_upload_file`. The function takes a `cloud_event` as input and prints various details from the event data, including event ID, type, bucket name, file name, metageneration, and creation time. Below the code, the terminal window displays the execution logs for the function, showing details like the ingress settings, revision, service account, and the current state of the function.

```

def arun_upload_file(cloud_event):
    event_type = cloud_event["type"]

    bucket = data["bucket"]
    name = data["name"]
    metageneration = data["metageneration"]
    timeCreated = data["timeCreated"]
    updated = data["updated"]

    print(f"Event ID: {event_id}")
    print(f"Event type: {event_type}")
    print(f"Bucket: {bucket}")
    print(f"File: {name}")
    print(f"Metageneration: {metageneration}")
    print(f"Created: {timeCreated}")

```

```

LOG_EXECUTION_ID: 'true'
ingressSettings: ALLOW_ALL
maxInstanceRequestConcurrency: 1
revision: arun-upload-file-00001-xev
service: projects/proven-record-481104-g3/locations/asia-south2/services/arun-upload-file
serviceAccountEmail: 219203465386-compute@developer.gserviceaccount.com
timeoutSeconds: 60
uri: https://arun-upload-file-zpv7tob5sa-em.a.run.app
state: ACTIVE
updateTime: '2025-12-30T10:45:23.680751279Z'
url: https://asia-south2-proven-record-481104-g3.cloudfunctions.net/arun_upload_file

```

STEP 6: Upload a File (Trigger the Function)

The screenshot shows the Google Cloud console interface for a storage bucket named `arun-file-upload-lab`. The bucket is located in the `asia-south2 (Delhi)` region, uses the `Standard` storage class, and has `Not public` access. The `Objects` tab is selected, showing a folder browser view. A file named `fact_transaction.csv` is listed with a size of 1.9 MB, type of `text/csv`, and created on 30 Dec 2025 at 16:17:23. The console also shows navigation options like `Create folder`, `Upload`, `Transfer data`, and `Other services`.

Name	Size	Type	Created	Storage class
fact_transaction.csv	1.9 MB	text/csv	30 Dec 2025, 16:17:23	Standard

STEP 7: Verify Cloud Logging

The screenshot shows the Google Cloud Platform console for a project named 'My First Project'. The 'Cloud Run' service is selected, and the 'Service details' page for 'arun-upload-file' is displayed. The service is in the 'asia-south2' region with a URL of 'https://asia-south2-proven-record-481104-g3.cloudfunctions.net/arun_upload_file'. The 'Observability' tab is active, showing a list of logs. The logs are filtered by 'Severity: Default' and show a series of events related to a file upload, including 'POST 200 130 B 16 ms', 'Event ID: 17608219282894175', 'Event type: google.cloud.storage.object.v1.finalized', 'Bucket: arun-file-upload-lab', 'File: fact_transaction.csv', 'Metageneration: 1', 'Created: 2025-12-30T10:47:23.448Z', and 'Updated: 2025-12-30T10:47:23.448Z'.

STEP 8: Verify Firestore Database Record

Open Firestore Console

GCP Console → Firestore → arundb

The screenshot shows the Google Cloud Platform console for a project named 'My First Project'. The 'Firestore' database is selected, and the 'Database arundb' page is displayed. The 'Panel view' is active, showing a list of collections. The 'arun_users_activity' collection is selected, and a document is being viewed. The document contains the following fields: 'email: "akhil@gmail.com"', 'employee-id: "1002"', and 'name: "akhil"'. The document ID is 'EwzJXPe3uc9TWqQLWpG'.

Practical Question 2 — Real-Time Event Ingestion & Analytics

Scenario : Build a real-time analytics pipeline using Pub/Sub and BigQuery.

Implementation:

I will first create a Pub/Sub topic named arun-user-activity to act as the real-time event source. Whenever a JSON message is published to this topic, a Cloud Function will be triggered automatically. The function will validate whether the message contains the required fields (event, user, ts). If valid, the event will be inserted into a BigQuery table for analytics. If invalid, the message will be logged and routed to a Dead Letter Queue (DLQ) Pub/Sub topic.

STEP 1: Create Pub/Sub Topics

1.1 Create main topic: user-activity

```
gcloud pubsub topics create arun-user-activity
```

1.2 Create Dead Letter Queue (DLQ) topic

```
gcloud pubsub topics create arun-user-activity-dlq
```

1.3 Create subscription for Cloud Function

```
gcloud pubsub subscriptions create arun-user-activity-sub \
  --topic=arun-user-activity
```

STEP 2: Create BigQuery Dataset & Table

2.1 Create BigQuery dataset

```
bq mk --location=asia-south2 arun_user_analytics
```

2.2 Create BigQuery table

```
bq mk \
  --table arun_user_analytics.user_events \
  event:STRING,user:STRING,ts:TIMESTAMP
```

```
Cloud Shell Editor
(proven-record-481104-g3) x +
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to proven-record-481104-g3.
Use 'gcloud config set project [PROJECT ID]' to change to a different project.
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub topics create arun-user-activity
Created topic [projects/proven-record-481104-g3/topics/arun-user-activity].
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub topics create arun-user-activity-dlq
Created topic [projects/proven-record-481104-g3/topics/arun-user-activity-dlq].
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub subscriptions create arun-user-activity-sub \
--topic=arun-user-activity
Created subscription [projects/proven-record-481104-g3/subscriptions/arun-user-activity-sub].
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ bq mk --location=asia-south2 arun_user_analytics
Dataset 'proven-record-481104-g3:arun_user_analytics' successfully created.
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ bq mk \
--table arun_user_analytics.user_events \
event:STRING,user:STRING,ts:TIMESTAMP
Table 'proven-record-481104-g3:arun_user_analytics.user_events' successfully created.
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub topics publish arun-user-activity --message "{\"event\":\"LOGIN\",\"user\":\"arun\",\"ts\":\"2025-01-01T10:00:00Z\"}"
messageIds:
- '17630934649482489'
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub subscriptions pull arun-user-activity-sub --limit=5 --auto-ack
DATA: {"event":"LOGIN","user":"arun",ts:2025-01-01T10:00:00Z}
MESSAGE_ID: 17630934649482489
ORDERING KEY:
ATTRIBUTES:
DELIVERY ATTEMPT:
ACK_STATUS: SUCCESS
```

Google Cloud My First Project pub

Navigation menu (.) topics

Pub/Sub	Topics
Topics	List Metrics
Subscriptions	Filter Filter topics
Snapshots	
Schemas	
Pub/Sub Lite	
Lite reservations	
Lite topics	
Lite Subscriptions	
Release notes	

Topic ID	Encryption key	Topic name	Retention	Ingest
arun-user-activity	Google-managed	projects/prove...	—	—
arun-user-activity-dlq	Google-managed	projects/prove...	—	—
eventarc-asia-south1-trigger-nab36x5f-621	Google-managed	projects/prove...	—	—
eventarc-asia-south2-processfilemetadata-098007-090	Google-managed	projects/prove...	—	—

STEP 3: Create Cloud Function Code

Create a folder

```
mkdir Handson2
cd Handson2
```

3.1 Python code (main.py)

```
import base64
import json
from google.cloud import bigquery, pubsub_v1

bq_client = bigquery.Client()
publisher = pubsub_v1.PublisherClient()

PROJECT_ID = "proven-record-481104-g3"
DLQ_TOPIC = f"projects/{PROJECT_ID}/topics/arun_user-activity-dlq"
TABLE_ID = f"{PROJECT_ID}.user_analytics_arun.user_events"

def arun_ingest_user_activity(event, context):
    try:
        # Decode message
        message = base64.b64decode(event['data']).decode('utf-8')
        data = json.loads(message)

        # Validate required fields
        if not all(k in data for k in ("event", "user", "ts")):
            raise ValueError("Missing required fields")

        # Insert into BigQuery
        rows = [{
            "event": data["event"],
            "user": data["user"],
            "ts": data["ts"]
        }]

        errors = bq_client.insert_rows_json(TABLE_ID, rows)
        if errors:
            raise RuntimeError(errors)

        print("Inserted event successfully")

    except Exception as e:
        print(f"Invalid message: {e}")
        publisher.publish(DLQ_TOPIC, event['data'])
```

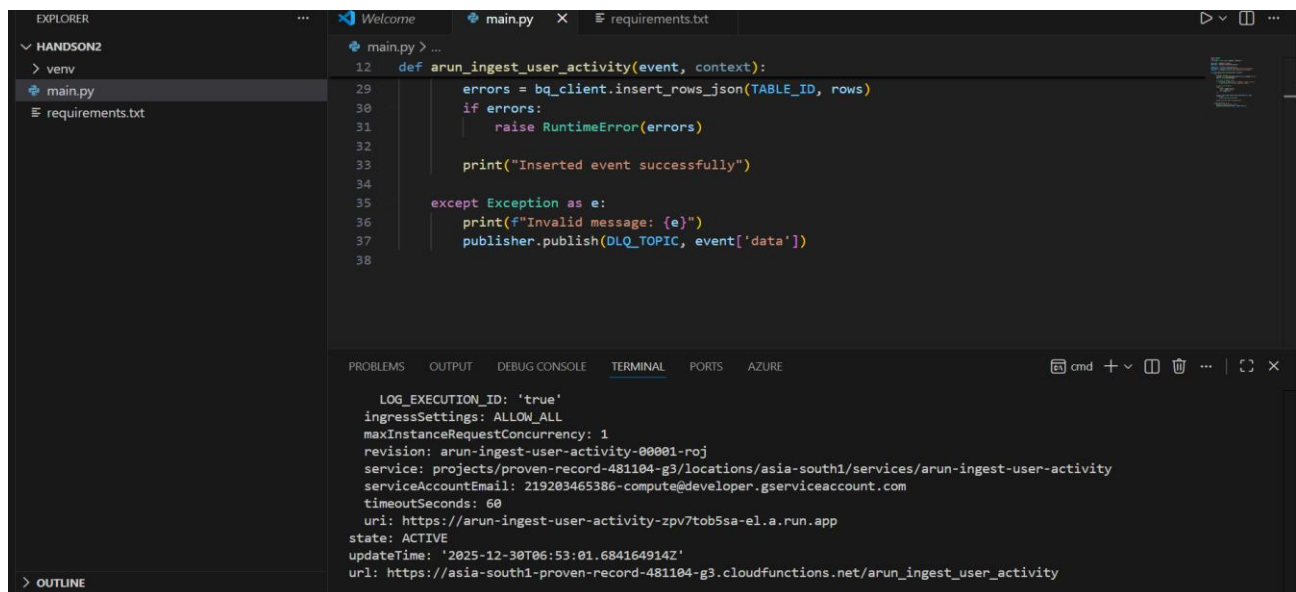
3.2 requirements.txt

google-cloud-pubsub

```
functions-framework==3.5.0
google-cloud-bigquery
google-cloud-firestore==2.22.0
```

STEP 4: Deploy Cloud Function

```
gcloud functions deploy arun_ingest_user_activity \
--gen2 \
--runtime python310 \
--source=. \
--trigger-topic arun-user-activity \
--region asia-south2 \
--entry-point arun_ingest_user_activity
```



The screenshot shows a VS Code editor with a file explorer on the left containing 'main.py' and 'requirements.txt'. The main editor displays the code for 'arun_ingest_user_activity' in 'main.py'. The terminal at the bottom shows the deployment details for the function.

```
12 def arun_ingest_user_activity(event, context):
29     errors = bq_client.insert_rows_json(TABLE_ID, rows)
30     if errors:
31         raise RuntimeError(errors)
32
33     print("Inserted event successfully")
34
35     except Exception as e:
36         print(f"Invalid message: {e}")
37         publisher.publish(DLQ_TOPIC, event['data'])
38
```

```
LOG_EXECUTION_ID: 'true'
ingressSettings: ALLOW_ALL
maxInstanceRequestConcurrency: 1
revision: arun-ingest-user-activity-00001-roj
service: projects/proven-record-481104-g3/locations/asia-south1/services/arun-ingest-user-activity
serviceAccountEmail: 219203465386-compute@developer.gserviceaccount.com
timeoutSeconds: 60
uri: https://arun-ingest-user-activity-zpv7tob5sa-el.a.run.app
state: ACTIVE
updateTime: '2025-12-30T06:53:01.684164914Z'
url: https://asia-south1-proven-record-481104-g3.cloudfunctions.net/arun_ingest_user_activity
```

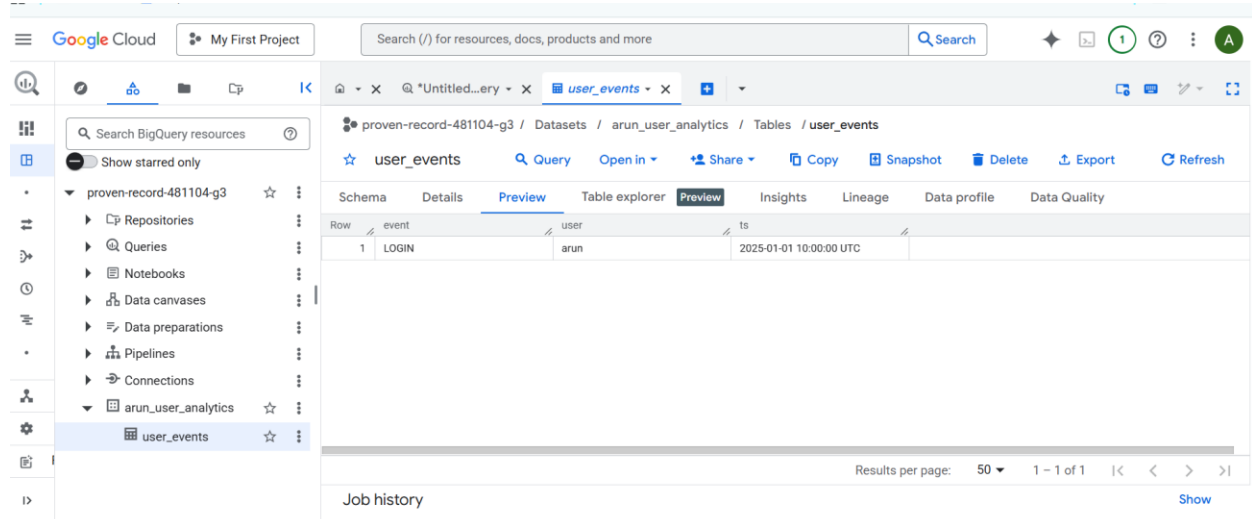
STEP 5: Publish Test Messages

5.1 Valid message

```
gcloud pubsub topics publish arun-user-activity --message
'{"event":"LOGIN","user":"arun","ts":"2025-01-01T10:00:00Z"}'
```

5.2 To Pull the data

```
gcloud pubsub subscriptions pull arun-user-activity-sub --limit=5 --auto-ack
```



5.2 Invalid message

```
gcloud pubsub topics publish arun-user-activity --message
{"event\":\"LOGIN\",\"user\":\"arun\"}
```

Missing ts → Routed to DLQ topic

STEP 6: Verify Results

6.1 Query BigQuery

```
SELECT * FROM `arun_user_analytics.user_events`;
```

The screenshot shows the Google Cloud BigQuery console. On the left is a navigation menu with options like 'Repositories', 'Queries', 'Notebooks', 'Data canvases', 'Data preparations', 'Pipelines', 'Connections', and 'arun_user_analytics'. The main area displays an 'Untitled query' with the SQL statement: `select * from arun_user_analytics.user_events;`. Below the query, a message states 'Query completed' and 'Using on-demand processing quota'. The 'Query results' section shows a table with one row of data.

Row	event	user	ts
1	LOGIN	arun	2025-01-01 10:00:00 UTC

At the bottom right of the results section, it says 'Results per page: 50' and '1 - 1 of 1'.

6.2 Check DLQ messages

gcloud pubsub subscriptions create arun-dlq-sub \
 --topic=arun-user-activity-dlq

The screenshot shows the Google Cloud Pub/Sub console for the topic 'arun-user-activity-dlq'. The left sidebar shows the 'Topics' section. The main area has tabs for 'Subscriptions', 'Snapshots', 'Metrics', 'Details', and 'Messages'. The 'Messages' tab is active, showing a table of messages. A message is visible with a publish time of '30 Dec 2025, 13:05:55' and a body of 'arun'. There is an 'Ack' button next to the message.

Publish time	Attribute keys	Message body	Ack
30 Dec 2025, 13:05:55	-	arun	Ack

On the right side of the console, there is a warning message: 'You do not have permission to view the permissions of the selected resource'.

Practical Question 3 — Event-Driven Data Ingestion

Scenario : Implement an event-driven data ingestion pipeline on Google Cloud using managed services.

Implementation : I will first create a Pub/Sub topic (arun-user-events) to act as the event source. Whenever a user activity event is published, it will trigger a Cloud Function (Gen 2) automatically. The Cloud Function will decode the Pub/Sub message and validate required fields (event, user, ts). If the message is valid, the function will insert the event data into the BigQuery table (user_activity). BigQuery will store the events in real time for analytics and reporting.

Step 1: Create Pub/Sub Topic

This topic will receive user activity events.

```
gcloud pubsub topics create arun-user-events
```

Step 2: Create subscription for Cloud Function

```
gcloud pubsub subscriptions create arun-user-events-sub --topic=arun-user-events
```

Step 3: Create BigQuery Dataset

Create a dataset to store analytics data.

```
bq mk --location=asia-south2 arun_analytics_ds
```

Step 4: Create BigQuery Table

Create the user_activity table with the required schema.

```
bq mk \  
--table arun_analytics_ds.user_activity \  
event:STRING,user:STRING,ts:TIMESTAMP
```

```
Cloud Shell Editor
(proven-record-481104-g3) x + v
vaddearun479@cloudshell:~ (proven-record-481104-g3)$ gcloud pubsub topics create arun-user-events
Created topic [projects/proven-record-481104-g3/topics/arun-user-events].
vaddearun479@cloudshell:~ (proven-record-481104-g3)$ gcloud pubsub subscriptions create arun-user-events-sub --topic=arun-user-events
Created subscription [projects/proven-record-481104-g3/subscriptions/arun-user-events-sub].
vaddearun479@cloudshell:~ (proven-record-481104-g3)$ bq mk --location=asia-south2 arun_analytics_ds
Dataset 'proven-record-481104-g3:arun_analytics_ds' successfully created.
vaddearun479@cloudshell:~ (proven-record-481104-g3)$ bq mk \
--table arun_analytics_ds.user_activity \
event:STRING,user:STRING,ts:TIMESTAMP
Table 'proven-record-481104-g3:arun_analytics_ds.user_activity' successfully created.
vaddearun479@cloudshell:~ (proven-record-481104-g3)$
```

Pub/Sub / Topics / Topic: arun-user-events

Pub/Sub

- Topics
- Subscriptions
- Snapshots
- Schemas

Pub/Sub Lite

- Lite reservations
- Lite topics
- Lite Subscriptions

Release notes

arun-user-events

Edit + Trigger Cloud Run function Import Delete

Learn Hide info panel

Topic name projects/proven-record-481104-g3/topics/arun-user-events

Tags

Subscriptions Snapshots Metrics Details Messages

Only subscriptions attached to this topic are displayed. A subscription captures the stream of messages published to a given topic. You can also stream messages to BigQuery or Cloud Storage by creating a subscription from a Cloud Dataflow job. [Learn more](#)

Create subscription Export

Filter Filter subscriptions

Subscription ID	Subscription name	Project
arun-user-events-sub	projects/proven-record-481104-...	proven-record-...

Permissions Labels Storage policy

You do not have permission to view the permissions of the selected resource

Step 5: Cloud Function (Gen 2) Code

main.py

```
import base64
import json
from google.cloud import bigquery

client = bigquery.Client()
table_id = "proven-record-481104-g3:arun-analytics_ds.user_activity"
```

```
def arun_ingest_user_event(event, context):
    pubsub_message = base64.b64decode(event['data']).decode('utf-8')
    data = json.loads(pubsub_message)

    rows = [{
        "event": data["event"],
        "user": data["user"],
        "ts": data["ts"]
    }]

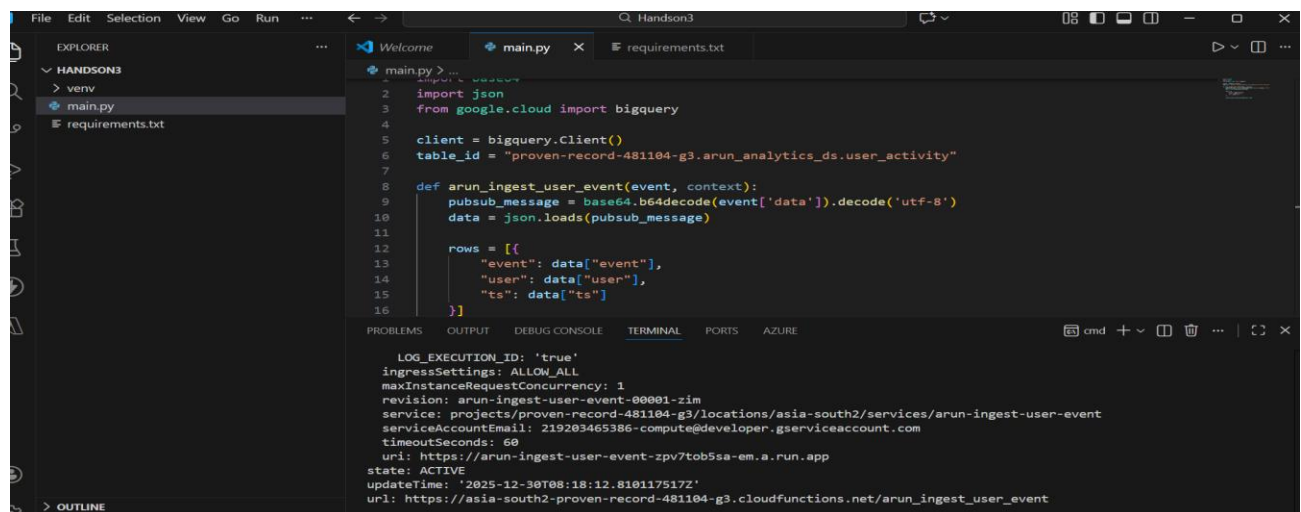
    client.insert_rows_json(table_id, rows)
```

requirements.txt

google-cloud-bigquery

Step 6: Deploy Cloud Function (Gen 2)

```
gcloud functions deploy arun_ingest_user_event \
--gen2 \
--runtime=python310 \
--region=asia-south2 \
--source=. \
--entry-point=arun_ingest_user_event \
--trigger-topic=arun-user-events
```



Step 7: Publish Test Message

```
gcloud pubsub topics publish arun-user-events \
--message='{ "event": "LOGIN", "user": "arun", "ts": "2025-01-01T10:00:00Z" }'
```

Step 8 : To Pull the data

```
gcloud pubsub subscriptions pull arun-user-events-sub --limit=5 --auto-ack
```

```
Cloud Shell Editor

(proven-record-481104-g3) x + -

vaddearun479@cloudshell:~ (proven-record-481104-g3) $ bq mk --location=asia-south2 arun_analytics_ds
Dataset 'proven-record-481104-g3:arun_analytics_ds' successfully created.
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ bq mk \
--table arun_analytics_ds.user_activity \
event:STRING,user:STRING,ts:TIMESTAMP
Table 'proven-record-481104-g3:arun_analytics_ds.user_activity' successfully created.
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub topics publish arun-user-events --message='{ "event": "LOGIN", "user": "arun", "ts": "2025-01-01T10:00:00Z" }'
messageId:
- '17633064879077215'
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub subscriptions pull arun-user-events-sub --limit=5 --auto-ack
DATA: { "event": "LOGIN", "user": "arun", "ts": "2025-01-01T10:00:00Z" }
MESSAGE ID: 17633064879077215
ORDERING KEY:
ATTRIBUTES:
DELIVERY ATTEMPT:
ACK STATUS: SUCCESS
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub topics publish arun-user-events --message='{ "event": "LOGIN", "user": "akhil", "ts": "2025-01-01T10:00:00Z" }'
messageId:
- '17632287194755689'
vaddearun479@cloudshell:~ (proven-record-481104-g3) $ gcloud pubsub subscriptions pull arun-user-events-sub --limit=5 --auto-ack
DATA: { "event": "LOGIN", "user": "akhil", "ts": "2025-01-01T10:00:00Z" }
MESSAGE ID: 17632287194755689
ORDERING KEY:
ATTRIBUTES:
DELIVERY ATTEMPT:
ACK STATUS: SUCCESS
vaddearun479@cloudshell:~ (proven-record-481104-g3) $
```

Step 9: Verify Data in BigQuery

```
SELECT * FROM `arun_analytics_ds.user_activity`
ORDER BY ts DESC;
```

The screenshot shows the Google Cloud BigQuery interface. The left sidebar contains a navigation menu with options like Repositories, Queries, Notebooks, Data canvases, Data preparations, Pipelines, Connections, analytics_ds, arun_analytics_ds, user_activity, and arun_user_analytics. The main panel displays the 'user_activity' table under the 'arun_analytics_ds' dataset. The table has columns for 'event', 'user', and 'ts'. The data is sorted by 'ts' in descending order. The table contains two rows: one for 'LOGIN' by 'akhil' at '2025-01-01 10:00:00 UTC' and another for 'LOGIN' by 'arun' at '2025-01-01 10:00:00 UTC'. The bottom of the interface shows a 'Job history' section with a 'Show' button.

Row	event	user	ts
1	LOGIN	akhil	2025-01-01 10:00:00 UTC
2	LOGIN	arun	2025-01-01 10:00:00 UTC

Google Cloud

My First Project

Search (/) for resources, docs, products and more

Search

1

Search BigQuery resources

Show starred only

Repositories

Queries

Notebooks

Data canvases

Data preparations

Pipelines

Connections

analytics_ds

arun_analytics_ds

user_activity

arun_user_analytics

Untitled query

Run

Download

Share

Schedule

Open in

More

Save

```
1 select * from arun_analytics_ds.user_activity
2 order by ts desc;
3
```

This query will process 43 B when run.

Using on-demand processing quota

Query results

Save results

Open in

Job information

Results

Visualisation

JSON

Execution details

Execution graph

Row	event	user	ts
1	LOGIN	akhil	2025-01-01 10:00:00 UTC
2	LOGIN	arun	2025-01-01 10:00:00 UTC

Results per page: 50

1 - 2 of 2

<

>

>|

Job history

Show